

Aula 05 — API REST: o CRUD completo

Curso de Formação de Programação Aplicada · Módulo 1 · FPA Labs

Duração: 2h · **Pré-requisito:** Aula 04

Objetivos de aprendizagem

Ao final desta aula o aluno deve conseguir:

1. Explicar o que é REST e por que a API é organizada por **recursos**.
2. Receber dados no corpo da requisição com `express.json()`.
3. Implementar as quatro operações do CRUD: criar, ler, atualizar e remover.
4. Escolher o status code correto para cada situação.
5. Validar a entrada e responder erros de forma consistente.

Plano de aula (2h)

Tempo	Bloco	Conteúdo
0:00–0:10	Aquecimento	Correção do exercício 5 da aula 04 (ordem das rotas)
0:10–0:30	Conceito	REST · recursos · o desenho de uma API bem feita
0:30–1:05	Código ao vivo	POST → PUT → PATCH → DELETE na API de Tarefas
1:05–1:15	Intervalo	
1:15–1:50	Mão na massa	Exercícios 1 a 4
1:50–2:00	Fechamento	O problema do arquivo de 300 linhas → prévia da aula 06

1. O que é REST

REST é um **conjunto de convenções** para desenhar APIs. Não é uma tecnologia nem uma biblioteca — é um acordo sobre como nomear as coisas, para que qualquer desenvolvedor entenda sua API sem precisar de manual.

A ideia central: **a URL identifica uma coisa (um recurso), o método diz o que fazer com ela.**

/tarefas → a coleção de tarefas
/tarefas/5 → a tarefa de id 5

Compare duas formas de desenhar a mesma API:

❌ Sem REST	✅ Com REST
GET /listarTarefas	GET /tarefas
GET /pegarTarefa?id=5	GET /tarefas/5
POST /criarNovaTarefa	POST /tarefas
POST /atualizarTarefa	PUT /tarefas/5
GET /deletarTarefa?id=5	DELETE /tarefas/5

Na coluna da esquerda, o verbo está no **nome** da rota e você precisa decorar cada uma. Na direita, o verbo está no **método HTTP** — e o padrão se repete para qualquer recurso: /usuarios, /produtos, /pedidos. Quem conhece uma, conhece todas.

As regras práticas de nomenclatura

1. **Substantivo, no plural, minúsculo:** /tarefas, não /Tarefa nem /getTarefas.
2. **Sem verbo na URL:** o verbo é o método HTTP.
3. **Hierarquia com barras:** /usuarios/3/tarefas = as tarefas do usuário 3.
4. **Filtro é query, não caminho:** /tarefas?concluida=true, não /tarefas/concluidas.

2. O CRUD e o desenho da nossa API

Operação	Método	Rota	Status de sucesso
Create	POST	/tarefas	201 Created
Read (lista)	GET	/tarefas	200 OK
Read (um)	GET	/tarefas/:id	200 OK
Update (todo)	PUT	/tarefas/:id	200 OK
Update (parte)	PATCH	/tarefas/:id	200 OK
Delete	DELETE	/tarefas/:id	204 No Content

Praticamente todo sistema que existe — banco, rede social, e-commerce, seu app de notas — é, no fundo, um CRUD com regras de negócio em cima. Dominar esse esqueleto é dominar 80% do back-end.

3. Recebendo dados: `express.json()`

Até agora só lemos dados da URL. Para criar uma tarefa, o cliente precisa **enviar** dados — e eles vêm no **corpo** da requisição.

```
const app = express();  
  
app.use(express.json()); // ⚠️ ESTA LINHA É OBRIGATÓRIA
```

`express.json()` é um **middleware**: uma função que roda **antes** das rotas e prepara a requisição. Este em específico lê o corpo em JSON e coloca o resultado em `req.body`.

⚠️ **O erro mais comum desta aula:** esquecer `app.use(express.json())` e receber `req.body === undefined`. Se acontecer, você vai passar 20 minutos procurando bug no lugar errado. Quando `req.body` vier `undefined`, **olhe primeiro para esta linha**.

Analogia: o middleware é o porteiro do prédio. Toda visita passa por ele antes de chegar ao apartamento. Ele pode conferir documento, anotar no livro, ou barrar. Aqui, ele "traduz" o pacote de JSON que chegou em um objeto JavaScript utilizável.

4. POST — criando

```
app.post("/tarefas", (req, res) => {
  const { titulo, prioridade } = req.body;

  // 1. VALIDAR – nunca confie no que chega de fora
  if (!titulo || titulo.trim() === "") {
    return res.status(400).json({ erro: "O campo 'titulo' é obrigatório" });
  }

  // 2. MONTAR – o servidor decide id, data e valores padrão. Nunca o cliente.
  const novaTarefa = {
    id: proximoId++,
    titulo: titulo.trim(),
    concluida: false,
    prioridade: prioridade ?? "media",
    criadaEm: new Date().toISOString(),
  };

  // 3. GUARDAR
  tarefas.push(novaTarefa);

  // 4. RESPONDER 201 com o recurso criado (o cliente precisa saber o id!)
  res.status(201).json(novaTarefa);
});
```

Três decisões importantes aqui:

- **Valide sempre.** Quem chama sua API pode ser um app com bug, alguém testando, ou um atacante. A regra é: *nunca confie na entrada*.
- **O servidor gera o `id`**, não o cliente. Deixar o cliente escolher o id é pedir para ter conflito e falha de segurança.
- **Devolva o objeto criado**, com o id. Sem isso o cliente não sabe o que fazer em seguida.

5. PUT e PATCH — atualizando

A diferença entre os dois confunde muita gente, mas é simples:

	PUT	PATCH
Significado	substitua o recurso inteiro	altere só o que eu mandei
Corpo	todos os campos	só os campos que mudam
Faltou um campo	ele é apagado/zerado	ele fica como estava

```
// PATCH – o mais usado no dia a dia
app.patch("/tarefas/:id", (req, res) => {
  const id = Number(req.params.id);
  const indice = tarefas.findIndex((tarefa) => tarefa.id === id);

  if (indice === -1) {
    return res.status(404).json({ erro: `Tarefa ${id} não encontrada` });
  }

  const { titulo, concluida, prioridade } = req.body;

  // Só troca o que veio; o resto permanece. Repare no spread da aula 02.
  const atualizada = {
    ...tarefas[indice],
    ...(titulo !== undefined && { titulo }),
    ...(concluida !== undefined && { concluida }),
    ...(prioridade !== undefined && { prioridade }),
  };

  tarefas[indice] = atualizada;
  res.json(atualizada);
});
```

O truque `...(condicao && { campo })` espalha o objeto só quando a condição é verdadeira. Se estranhar, escreva com `if` — funciona igual e é mais legível para quem está começando.

⚠ **Cuidado com `if (concluida)`**. Se o cliente mandar `concluida: false`, essa condição é falsa e o campo é ignorado — bug clássico. Use sempre `!== undefined` para checar presença.

6. DELETE — removendo

```
app.delete("/tarefas/:id", (req, res) => {
  const id = Number(req.params.id);
  const indice = tarefas.findIndex((tarefa) => tarefa.id === id);

  if (indice === -1) {
    return res.status(404).json({ erro: `Tarefa ${id} não encontrada` });
  }

  tarefas.splice(indice, 1);

  // 204 = "deu certo, e não tenho nada para te devolver"
  res.status(204).end();
});
```

Por que `204` e não `200`? Porque o recurso deixou de existir — não há o que retornar. Alguns times preferem `200` com o objeto removido. Ambos são aceitáveis; o importante é **ser consistente** dentro da mesma API.

7. Validação: um padrão que escala

Espalhar `if` pelas rotas funciona com 2 campos. Com 8, vira bagunça. Um passo intermediário simples é juntar todos os erros de uma vez:

```
function validarTarefa(dados) {
  const erros = [];

  if (!dados.titulo || dados.titulo.trim() === "") {
    erros.push("titulo é obrigatório");
  } else if (dados.titulo.length > 100) {
    erros.push("titulo deve ter no máximo 100 caracteres");
  }

  const prioridadesValidas = ["baixa", "media", "alta"];
  if (dados.prioridade && !prioridadesValidas.includes(dados.prioridade)) {
    erros.push(`prioridade deve ser: ${prioridadesValidas.join(", ")}`);
  }

  return erros;
}

// na rota:
const erros = validarTarefa(req.body);
if (erros.length > 0) {
  return res.status(400).json({ erros });
}
```

Devolver **todos** os erros de uma vez é muito melhor para quem consome a API: em vez de corrigir um campo, tentar de novo, descobrir outro erro, corrigir... o cliente vê tudo de uma vez.

Na aula 06 vamos levar isso a sério, com uma biblioteca de validação.

8. Consistência de respostas

Escolha um formato e **use sempre o mesmo**. Sugestão do curso:

```
// sucesso com lista
{ "total": 3, "dados": [ ... ] }

// sucesso com um item
{ "id": 1, "titulo": "... " }

// erro
{ "erro": "Tarefa 99 não encontrada" }

// erro de validação (vários)
{ "erros": ["titulo é obrigatório", "prioridade inválida"] }
```

Não existe formato "certo". Existe formato **consistente**. Uma API onde cada rota responde de um jeito diferente é um pesadelo para o front-end — que, aliás, será você mesmo na aula 09.

9. Testando o CRUD inteiro

Com o `api.http`, teste o ciclo completo:

```
### 1. criar
POST http://localhost:3000/tarefas
Content-Type: application/json

{ "titulo": "Estudar REST", "prioridade": "alta" }

### 2. conferir que apareceu na lista
GET http://localhost:3000/tarefas

### 3. marcar como concluída
PATCH http://localhost:3000/tarefas/5
Content-Type: application/json

{ "concluida": true }

### 4. apagar
DELETE http://localhost:3000/tarefas/5

### 5. conferir que sumiu (deve dar 404)
GET http://localhost:3000/tarefas/5
```

Teste **também** os caminhos infelizes: criar sem título, atualizar id inexistente, mandar prioridade inválida. Um dev júnior testa o caminho feliz; um dev sênior testa o resto.

Lembre: os dados vivem em memória. Ao reiniciar o servidor, tudo volta ao estado inicial. Na aula 07 resolvemos isso com um banco de dados.

10. Ferramentas novas desta aula

`express.json()`

O que é: um middleware — uma função que roda **antes** das rotas. Este lê o corpo da requisição em JSON e coloca o resultado pronto em `req.body`.

```
app.use(express.json());
```

Sem essa linha, `req.body` é `undefined`. É a causa nº 1 de "a API não está recebendo meus dados".

O spread condicional

```
...(titulo !== undefined && { titulo })
```

Parece críptico, mas é simples quando destrinchado:

Se <code>titulo</code> foi enviado	O <code>&&</code> devolve	Espalhar isso resulta em
sim	<code>{ titulo: "novo" }</code>	o campo é adicionado
não	<code>false</code>	nada acontece

Espalhar `false` dentro de um objeto é inofensivo — o JavaScript simplesmente ignora. Se achar confuso, escreva com `if`: funciona igual e é mais legível para quem está começando.

`.push()` e `.splice()` — os que alteram o array

No back-end, com dados em memória, mutar é aceitável:

```
tarefas.push(nova);           // adiciona no fim; devolve o novo tamanho
tarefas.splice(indice, 1);    // remove 1 item a partir da posição indicada
tarefas[indice] = atualizada;  // substitui na posição
```

⚠️ `slice` × `splice` — uma letra de diferença, comportamentos opostos:

	O que faz	Altera o original?
<code>.slice(inicio, fim)</code>	copia um pedaço	❌ não
<code>.splice(inicio, quantos)</code>	corta do original	🔴 sim

Guarde isso: na aula 08, no React, **nenhum** dos métodos que mutam pode ser usado.

`.findIndex()` no CRUD

```
const indice = tarefas.findIndex((tarefa) => tarefa.id === id);
if (indice === -1) return res.status(404).json({ erro: "não encontrada" });
```

Usamos `findIndex` (e não `find`) quando precisamos da **posição** para depois alterar ou remover o item naquele lugar.

⚠️ Sempre teste com `=== -1`. Um `if (!indice)` daria bug com o primeiro item da lista, porque a posição `0` é "falsa" em JavaScript.

`new Date().toISOString()`

```
new Date(); // objeto com a data e hora de agora
new Date().toISOString(); // "2026-04-01T13:45:30.123Z"
```

Guarde datas sempre nesse formato. Três razões:

1. Ordena corretamente como texto (`"2026-01" < "2026-02"`).
2. Não tem ambiguidade entre dia e mês.
3. É um padrão internacional, entendido por qualquer sistema.

`"01/04/2026"` é 1º de abril no Brasil e 4 de janeiro nos Estados Unidos — por isso esse formato nunca é usado para **armazenar**. Ele serve só para **mostrar** ao usuário, e aí você usa

`.toLocaleDateString("pt-BR")`.

`Object.keys()` para detectar objeto vazio

```
Object.keys({ a: 1, b: 2 }); // ["a", "b"]
Object.keys({}); // []

if (Object.keys(req.body).length === 0) {
  return res.status(400).json({ erro: "informe pelo menos um campo" });
}
```

Um `PATCH` com corpo vazio não faz sentido — vale recusar com `400`.

`.some()` para checar duplicidade

```
const jaExiste = filmes.some(
  (filme) => filme.titulo.toLowerCase() === novoTitulo.toLowerCase()
);
```

`.some()` devolve `true` se **pelo menos um** item atender à condição. É a forma mais direta de implementar a regra "não pode repetir" — e repare no `.toLowerCase()` dos dois lados, para que "Bacurau" e "bacurau" contem como o mesmo título.

Recapitulação em 3 frases

1. REST: a URL diz **qual coisa**, o método HTTP diz **o que fazer** com ela.
2. `express.json()` é obrigatório para ler `req.body` — é o primeiro lugar a olhar quando ele vier `undefined`.
3. Valide toda entrada, deixe o servidor gerar o id, e responda o status correto: 201 ao criar, 204 ao apagar, 400 em dado inválido, 404 em recurso inexistente.

Tarefa para a próxima aula

Exercícios 1 a 4 de [exercicios/README.md](#).

Na próxima aula

Seu `server.js` já deve estar com umas 150 linhas. Imagine com 20 rotas. Na aula 06 vamos **organizar isso em camadas** — do jeito que se faz em empresa.